CHAPTER 28

Neural Network Foundations

Building on the inspiration of the biological neuron, the artificial neural network (ANN) is a society of connectionist agents that learn and transfer information from one artificial neuron to the other. As data transfers between neurons, a hierarchy of representations or a hierarchy of features is learned, hence the name deep representation learning or deep learning.

The Architecture

An artificial neural network is composed of

- An input layer
- Hidden layer(s)
- An output layer

E. Bisong, *Building Machine Learning and Deep Learning Models on Google Cloud Platform*,
https://doi.org/10.1007/978-1-4842-4470-8_28

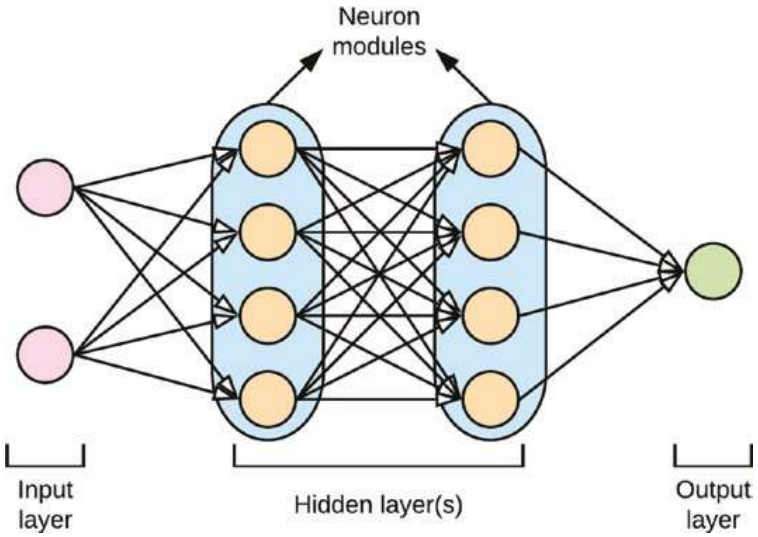


Figure 28-1. *Neural network architecture*

The input layer receives information from the features of the dataset, after which some computation takes place, and information that captures the learned patterns of the data is propagated across the hidden layer(s) with hopes to improve the learned patterns.

The hidden layer(s) is where the workhorse of deep learning occurs. The hidden layer(s) can consist of multiple neuron modules as shown in Figure 28-1. Each hidden network layer learns a more sophisticated set of feature representations. The decision on the number of neurons in a layer (network width) and the number of hidden layers (network depth) which forms the network topology is a design choice when training deep learning networks. The techniques for training a deep neural network are discussed in the next chapter.

CHAPTER 29

Training a Neural Network

This chapter gives an overview of the techniques for training a deep neural network. Here, we briefly discuss

- How learned information flows through a neural network
- The role of the cost function at the output layer of the network
- One-hot encoding and the softmax activation function for determining class membership at the output layer of a classification problem
- The backpropagation algorithm for improving the learned parameters of the network
- Activation functions that enable the neural network to learn non-linear patterns

In this chapter, as we discuss the methods involved in training a neural network, we will use the example of a classification problem with two possible outputs. In designing a neural network, the number of neurons in the input layer is typically the number of features of the dataset, while the number of neurons in the output layer is the number of classes in the target variable that the neural network is learning to classify.

As illustrated in Figure 29-1, the dataset features are the inputs to the neural network, while the classes in the target variable determine the number of output neurons. In this example, the network learns two classes, 0 and 1.

E. Bisong, *Building Machine Learning and Deep Learning Models on Google Cloud Platform*,
https://doi.org/10.1007/978-1-4842-4470-8_29

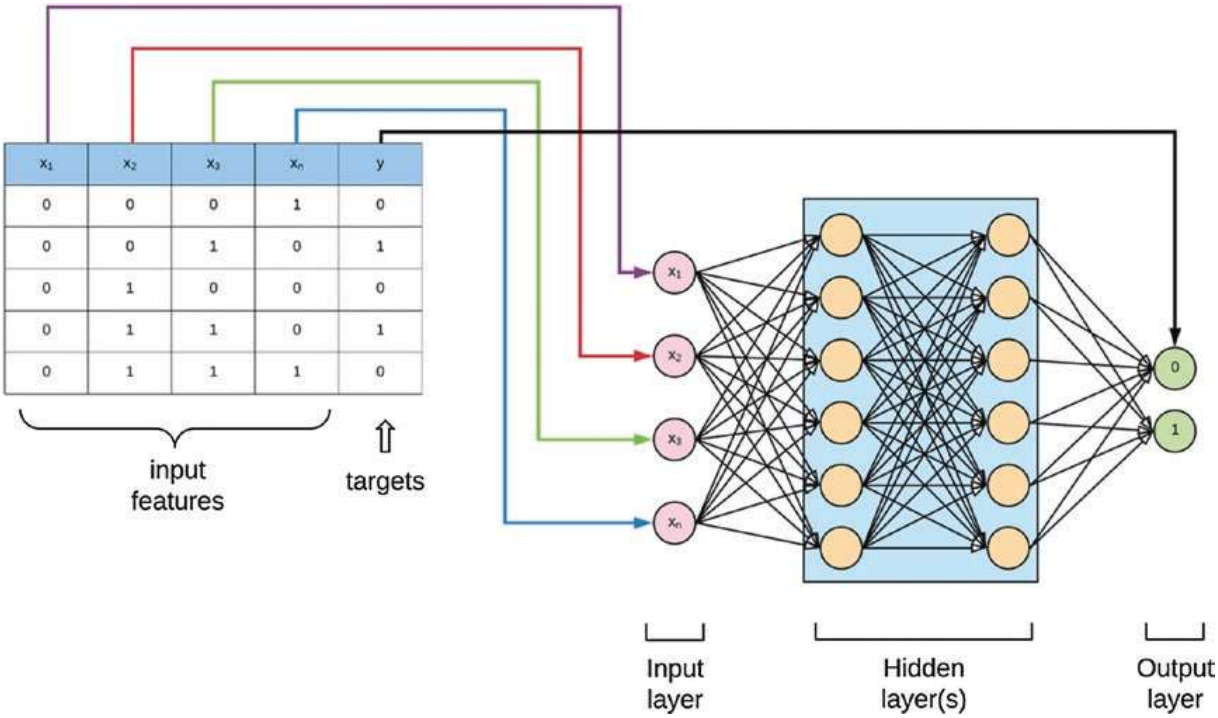


Figure 29-1. *Defining a neural network from a dataset*

A weight (also called parameter) is assigned to every neuron. The weights of neurons in a neural layer are multiplied by their inputs and then passed through an activation function (to be discussed in this chapter) for which the outputs are the inputs to the neurons in the next neural layer of the network (see Figure 29-2). This procedure is repeated as information of what the neural network is trying to learn moves from one layer of the network to another. Every neuron layer also has a bias neuron (typically set to 1) that controls the weighted sum. This is similar to the bias term in the logistic regression model.

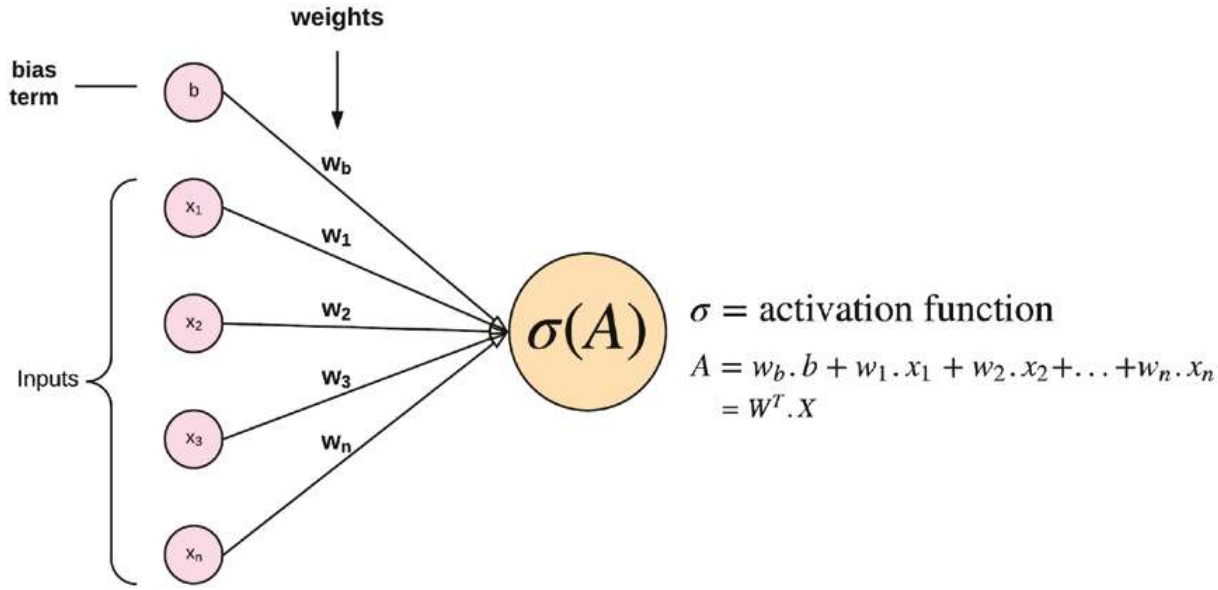


Figure 29-2. *Information flowing from a previous neural layer to a neuron in the next layer*

The weights are initialized as random values that are later adjusted as the network begins to learn using the backpropagation algorithm (to be discussed in this chapter). In summary, the outputs (or activations) of the neurons in the neural network layers are determined by the sum of the weight times the outputs plus the bias term of the neurons in the previous layer acted upon by a non-linear *activation function* (see Figure 29-2). This move is called the feedforward learning algorithm.

However, the output of the feedforward pass through the network may most likely result in an incorrect classification. The errors made from the feedforward procedure are later adjusted using the backpropagation algorithm (to be discussed). To evaluate the performance of the neural network, we define a cost function or loss function (similar to other machine learning algorithms) that captures the quality of the prediction made by the network.

The goal of the neural network is to minimize the cost function. Two commonly used cost functions are the squared error cost function for regression problems and the softmax cross-entropy cost function for classification problems.

Cost Function or Loss Function

The squared error cost function (also known as the mean squared error) finds the sum of the squared difference between the estimated target and the actual target for a real-valued problem, while the cross-entropy cost function finds the difference between the predicted class from the probability estimates of the actual class label in a classification problem.

Regardless of the cost function used, when the error loss is small, we say that the cost is minimized. In Figure 29-3, the correct output of the example passed into the network is **2.3**. After the output values are evaluated from the feedforward training, the squared error cost function is used to assess the quality of the network's output.

Remember that the MSE finds the average cost over all the data samples in the training dataset. In the example illustrated in Figure 29-3, we used just one data sample to demonstrate how the cost function works.

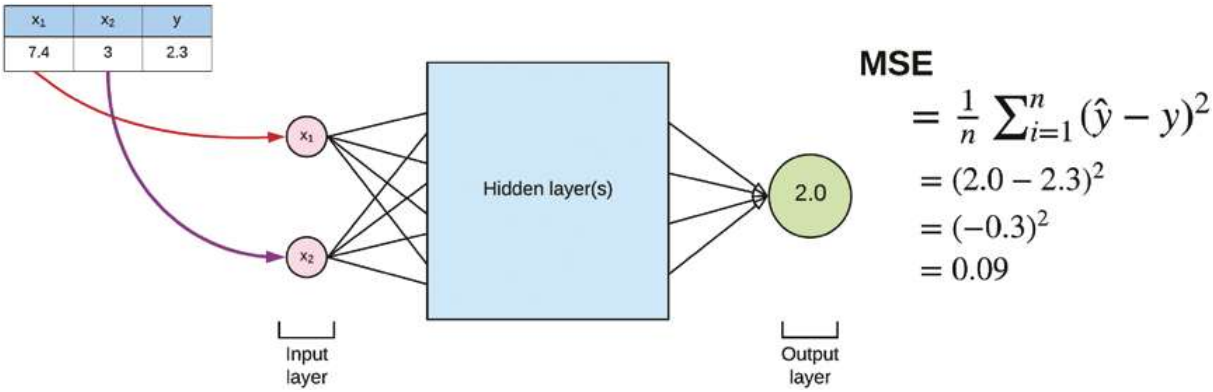
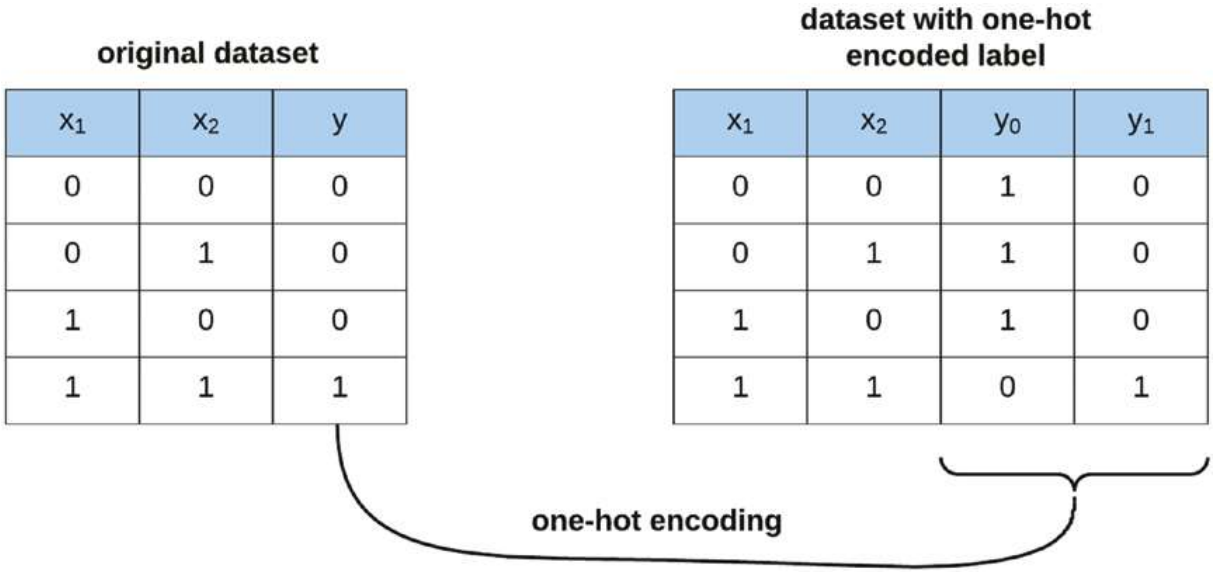


Figure 29-3. *MSE estimate of the neural network*

One-Hot Encoding

In a classification problem, one-hot encoding is the process of transforming the class labels of the target variable into a matrix of binary variables. The one-hot encoder assigns 1 when the output belongs to a particular class and 0 otherwise. An illustration of one-hot encoding is shown in Figure 29-4.



x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

x_1	x_2	y_0	y_1
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 29-4. *One-hot encoding*

In the final layer of the neural network, just before the output layer, an activation function called the softmax (same as discussed under "Logistic Regression") is applied to transform the activations to the probability that the example belongs to one of the output classes.

The purpose of applying one-hot encoding to the labels of the dataset is to represent the output as a vector of distinct classes with the probability that an example in the training dataset belongs to any one of the output categories.

The Backpropagation Algorithm

Backpropagation is the process by which we train the neural network to improve its prediction accuracy. To train the neural network, we need to find a mechanism for adjusting the weights of the network; this in turn affects the value of the activations within each neuron and consequently updates the value of the predicted output layer. The first time we run the feedforward algorithm, the activations at the output layer are most likely incorrect with a high error estimate or cost function.

The goal of backpropagation is to repeatedly go back and adjust the weights of each preceding neural layer and perform the feedforward algorithm again until we minimize the error made by the network at the output layer (see Figure 29-5).

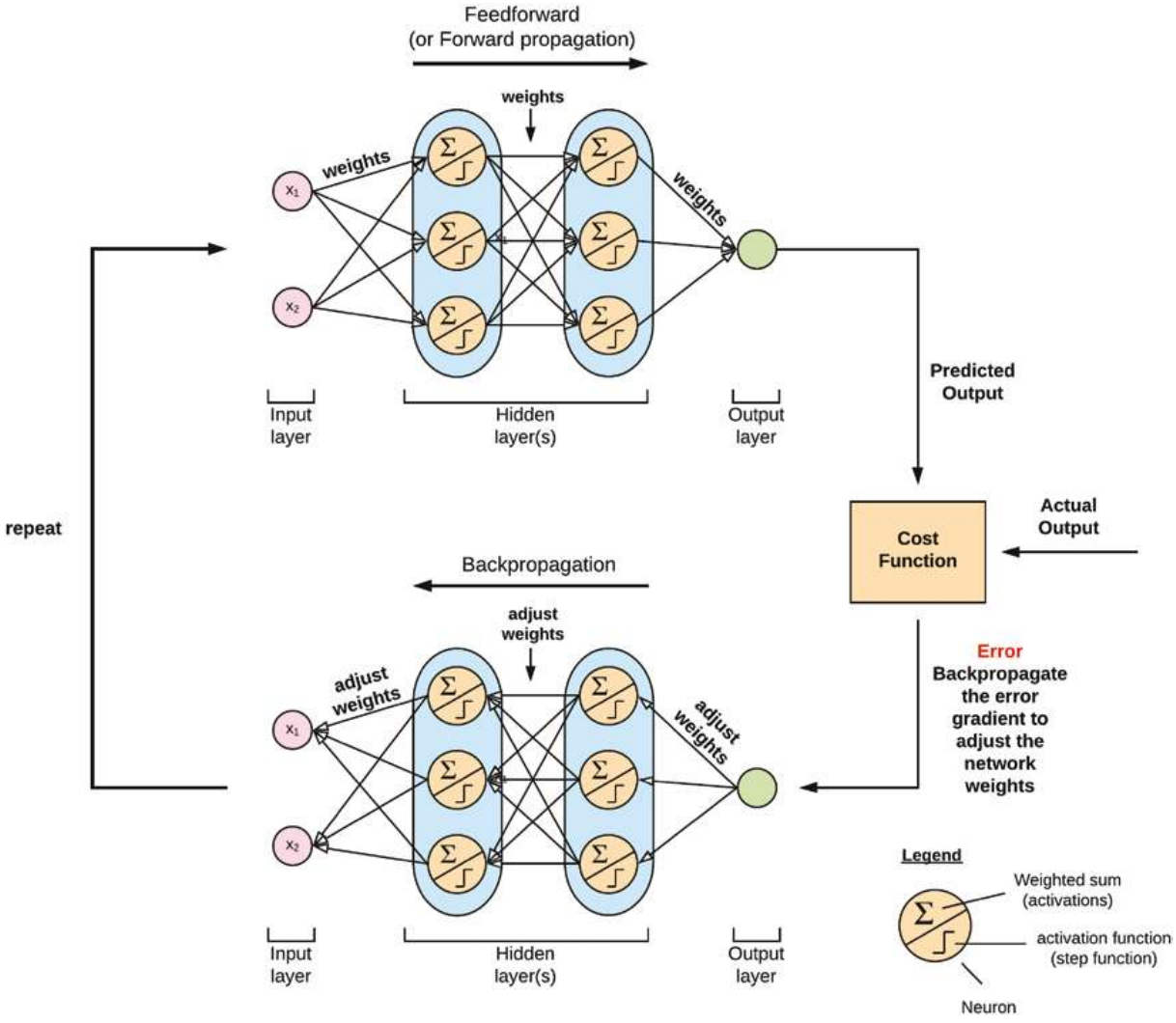


Figure 29-5. *Backpropagation*

The backpropagation algorithm works by computing the cost function at the output layer by comparing the predicted output of the neural network with the actual outputs from the dataset. It then employs gradient descent (earlier discussed in Chapter 16) to calculate the gradient of the cost function using the weights of the neurons at each successive layer and update the weights propagating back through the network.

Activation Functions

Up till now, we have mentioned activation functions. Now let's go a bit deeper into what activation functions are and why do we have them.

Activation functions act on the weighted sum in the neuron (which is nothing more than the weighted sum of weights and their added bias) by passing it through a non-linear function to decide if that neuron should fire (propagate) its information or not to the succeeding neural layers.

In other words, the activation function determines if a particular neuron has the information to result in a correct prediction at the output layer for an observation in the training dataset. Activation functions are analogous to how neurons communicate and transfer information in the brain, by firing when the activation goes above a particular threshold value.

These activation functions are also called non-linearities because they inject non-linear capabilities to our network and can learn a mapping from inputs to output for a dataset whose fundamental structure is non-linear. An illustration of passing the weighted sum of weights and biases through an activation function is shown in Figure 29-6.

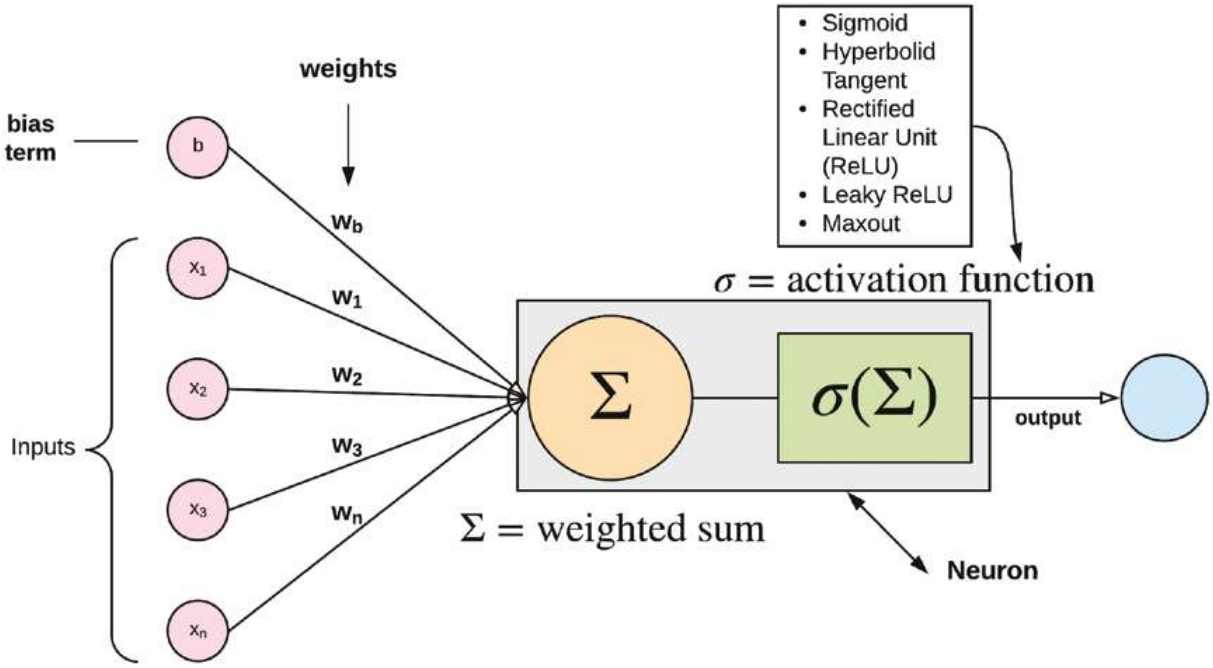


Figure 29-6. *Activation function*

The following are examples of activation functions used in a neural network:

- Sigmoid
- Hyperbolic tangent (tanh)

- Rectified linear unit (ReLU)
- Leaky ReLU
- Maxout

Let's briefly examine them.

Sigmoid

The sigmoid function illustrated in Figure 29-7 is a non-linear function that brings (or squashes) the activations to fall within a range of 0 and 1. This brings large negative and positive numbers to 0 and 1, respectively. The neurons typically begin firing when the function output is above a threshold of 0.5.

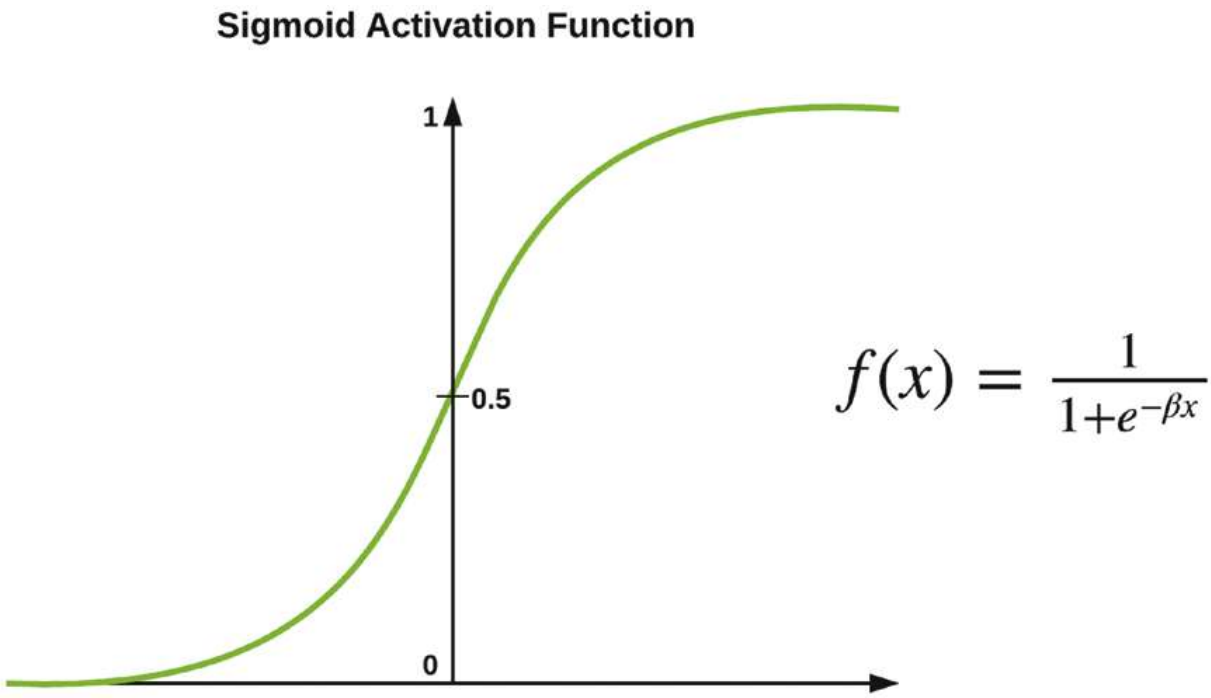


Figure 29-7. *Sigmoid activation function*

However, a significant drawback of the sigmoid function is its susceptibility to a phenomenon called exploding and vanishing gradients. In the process of optimizing the weights of the network during backpropagation, the gradients can become disproportionately small or large with their activations concentrated at either 0 or 1.

When this happens, we say that the gradients have saturated. Hence, further multiplication via backpropagation causes the gradient to either vanish or explode; and as a result, the affected neurons become dead and transfer no information across the network, thus negatively affecting training.

Another drawback is that the outputs of the function are not zero-centered. As a consequence, during backpropagation, the gradients can either become all positive or all negative. This has a negative effect in minimizing the function objective (i.e., the cost function).

Hyperbolic Tangent (tanh)

The hyperbolic tangent illustrated in Figure 29-8 improves on the sigmoid function by bordering its output within a range of −1 and 1. So, while it still suffers from the exploding and vanishing gradient problem, its outputs are now zero-centered. From the formula, the reader will observe that tanh is merely a scaled sigmoid function.

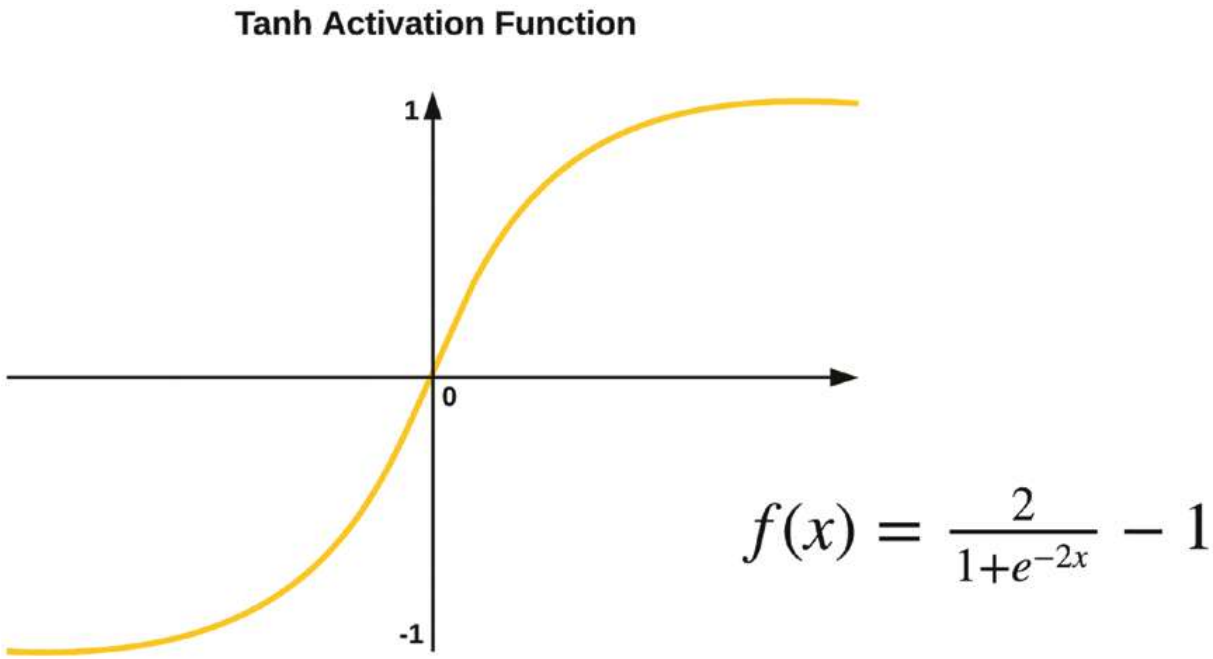


Figure 29-8. *The hyperbolic tangent activation function*

Rectified Linear Unit (ReLU)

The rectified linear unit or ReLU activation function is illustrated in Figure 29-9 and works by setting the activation to 0 for values, x, less than 0 and a linear slope of 1 when values, x, are greater than 0.

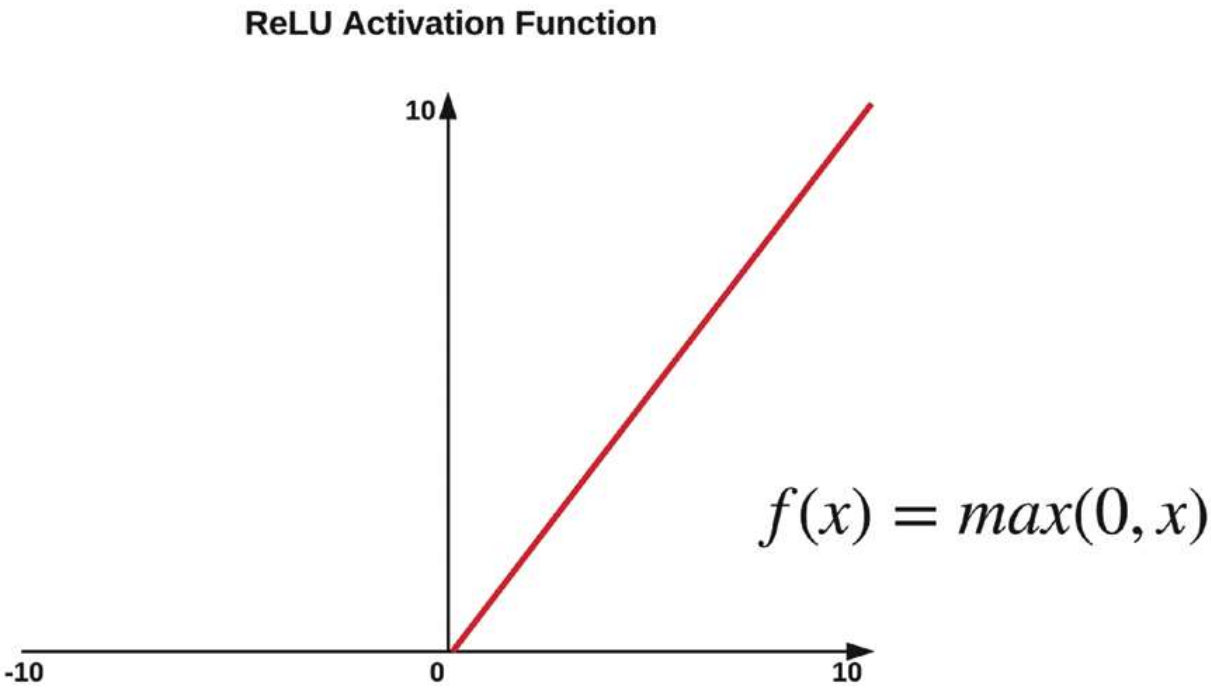


Figure 29-9. *ReLU activation function*

ReLU offers a vast improvement on the tanh and sigmoid activation functions by greatly mitigating the vanishing and exploding gradient problem. However, some gradients can still die out during backpropagation with a large learning rate. However, with a well-defined learning rate, we should not have a problem.

Leaky ReLU

Leaky ReLU is another activation function that is proposed to solve the case of some neurons completely dying out in ReLU by avoiding zero gradients. Leaky ReLU is illustrated in Figure 29-10. The function works by setting the activation to a small negative slope when the value $x < 0$.

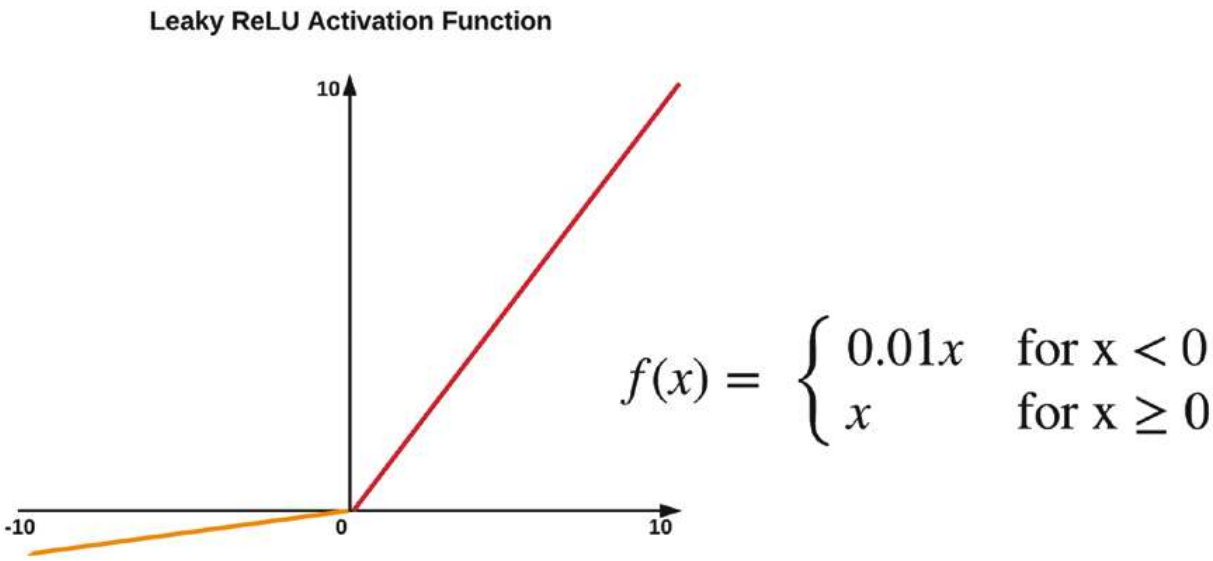


Figure 29-10. *Leaky ReLU activation function*

Maxout

The Maxout activation function generalizes the ReLU and leaky ReLU functions and hence takes advantage of the efficiency of ReLU while avoiding its pitfalls of some neurons dying out. In any case, a trade-off needs to be made, because Maxout increases the parameter size of each neuron during training.

As a rule of thumb, different types of activation functions are not mixed in the same network. Also, ReLU is typically used for the hidden layers, and the softmax activation is used for classification problems at the output layer since this layer returns a probability of membership of a particular class.

This chapter provided an overview on how to train a predictive model using neural networks. This chapter ends Part 5 on introducing deep learning. The chapters in Part 6 will cover deep learning algorithms and their implementation with TensorFlow and Keras.